UNIDAD 10
MANIPULACIÓN Y TRANSACCIONALIDAD

FICHA TÉCNICA DE LA UNIDAD DIDÁCTICA

Unidad Didáctica	Unidad Didáctica 10: Manipulación de Datos y Transaccionalidad en MySQL
Resultados de Aprendizaje	**RA5:** Realiza operaciones de manipulación de datos de forma segura, garantizando la integridad, consistencia y persistencia de la información según las especificaciones del diseño.
Criterios de Evaluación	**CE5.a:** Se han identificado los comandos y herramientas para la inserción, modificación y eliminación de datos. **CE5.b:** Se han realizado operaciones de inserción de registros en una o varias tablas de forma simultánea. **CE5.c:** Se han modificado registros de forma masiva o selectiva aplicando criterios de filtrado. **CE5.d:** Se han eliminado registros garantizando la integridad referencial. **CE5.e:** Se ha descrito el concepto de transacción y sus propiedades ACID. **CE5.f:** Se han ejecutado transacciones utilizando comandos TCL de confirmación, deshacer y puntos de retorno. **CE5.g:** Se han analizado los problemas de concurrencia y configurado los niveles de aislamiento adecuados en el motor InnoDB.
Tiempo Estimado	18 Horas (Planificado para 3 semanas de sesiones teorico prácticas)

ÍNDICE DE CONTENIDOS

[1. INTRODUCCIÓN Y CONTEXTUALIZACIÓN]

[2. LENGUAJE DE MANIPULACIÓN DE DATOS (DML)]

```
2.1 Inserción de
Datos: Sentencia INSERT
2.2 Modificación
de Datos: Sentencia UPDATE
2.3 Eliminación
de Datos: DELETE y TRUNCATE
2.4
Operaciones de Manipulación e Integridad Referencial
```

3. TRANSACCIONES

3.1 Concepto de Transacción y Propiedades ACID

3.2 Control del AUTOCOMMIT en MySQL

3.3 Comandos TCL: START TRANSACTION, COMMIT y ROLLBACK

3.4 Control Fino con Puntos de Retorno (SAVEPOINT)

4. ACCESO CONCURRENTE Y NIVELES DE AISLAMIENTO EN INNODB

4.1 Fenómenos de Lectura Concurrente

4.2 Niveles de Aislamiento

5. TRANSACCIONES EN PROCEDIMIENTOS ALMACENADOS

5.1 Gestión de Excepciones y Reversión Automática

1. INTRODUCCIÓN Y CONTEXTUALIZACIÓN

El módulo de Gestión de Bases de Datos tiene como finalidad dotar al administrador de sistemas de las capacidades necesarias para mantener la integridad, disponibilidad y seguridad del activo más importante de cualquier organización: sus datos.

Una vez que se han diseñado físicamente las estructuras de almacenamiento (tablas, restricciones y relaciones) y que se ha aprendido a extraer información compleja a través de consultas declarativas, es imperativo dominar el ciclo de vida del dato. Esto incluye desde su entrada en el sistema hasta su posterior actualización o borrado bajo un entorno transaccional multitarea.

Esta unidad se conecta de forma directa con la *Unidad 5 (Recuperación de Información)* y la *Unidad 12 (Triggers y Programación de Procedimientos)*. Desde la perspectiva de un administrador de sistemas de red, el control transaccional es la última barrera de defensa para evitar la corrupción de datos ante caídas de servicio, fallos de red o inconsistencias físicas de almacenamiento en sistemas en producción.

2. LENGUAJE DE MANIPULACIÓN DE DATOS (DML)

Para realizar los ejemplos de esta unidad de trabajo, utilizaremos el esquema relacional de la base de datos de **Jardinería / Pedidos** definido a continuación:

```
-- Creación de la base de datos con codificación utf8mb4 recomendada
para SysAdmins
CREATE DATABASE IF NOT EXISTS jardineria_pedidos
CHARACTER SET utf8mb4
COLLATE utf8mb4_spanish_ci;
USE jardineria_pedidos;
-- Tabla: CLIENTE
CREATE TABLE cliente (
codigo_cliente INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
nombre_cliente VARCHAR(50) NOT NULL,
limite_credito DECIMAL(15, 2) NOT NULL DEFAULT 1000.00
) ENGINE=InnoDB;
-- Tabla: PRODUCTO
CREATE TABLE producto (
codigo_producto VARCHAR(15) PRIMARY KEY,
nombre VARCHAR(70) NOT NULL,
cantidad_en_stock SMALLINT UNSIGNED NOT NULL DEFAULT 0,
precio_venta DECIMAL(15, 2) NOT NULL
) ENGINE=InnoDB;
-- Tabla: PEDIDO
CREATE TABLE pedido (
codigo_pedido INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
fecha_pedido DATE NOT NULL,
fecha_esperada DATE NOT NULL,
fecha_entrega DATE DEFAULT NULL,
estado VARCHAR(15) NOT NULL DEFAULT "Pendiente",
codigo_cliente INT UNSIGNED NOT NULL,
CONSTRAINT fk_pedido_cliente FOREIGN KEY (codigo_cliente)
REFERENCES cliente (codigo_cliente)
) ENGINE=InnoDB;
-- Tabla: DETALLE_PEDIDO
CREATE TABLE detalle_pedido (
codigo_pedido INT UNSIGNED,
codigo_producto VARCHAR(15),
cantidad INT UNSIGNED NOT NULL,
precio_unidad DECIMAL(15, 2) NOT NULL,
PRIMARY KEY (codigo_pedido, codigo_producto),
```

```
CONSTRAINT fk_detalle_pedido FOREIGN KEY (codigo_pedido)
REFERENCES pedido (codigo_pedido) ON DELETE CASCADE,
CONSTRAINT fk_detalle_producto FOREIGN KEY (codigo_producto)
REFERENCES producto (codigo_producto) ON DELETE RESTRICT
) ENGINE=InnoDB;
-- Tabla: PAGO
CREATE TABLE pago (
codigo_cliente INT UNSIGNED,
id_transaccion VARCHAR(26) PRIMARY KEY,
fecha_pago DATE NOT NULL,
total DECIMAL(15, 2) NOT NULL,
CONSTRAINT fk_pago_cliente FOREIGN KEY (codigo_cliente)
REFERENCES cliente (codigo_cliente)
) ENGINE=InnoDB;
```

2.1 Inserción de Datos: Sentencia INSERT

La sentencia INSERT permite añadir nuevos registros (filas) a una tabla existente en la base de datos.

🔗 **Documentación Oficial: Más información en [Documentación Oficial de MySQL sobre la Sentencia INSERT]**

Sintaxis Básica de Inserción Posicional e Inserción Explícita

La buena práctica en administración exige siempre detallar la lista de columnas destino para evitar que cambios futuros en el esquema de la tabla rompan los scripts de automatización:

```
-- Práctica Recomendada: Inserción explícita especificando
columnas
INSERT INTO cliente (nombre_cliente, limite_credito)
VALUES ('Jardines Celia S.L.', 5000.00);
-- Inserción múltiple de alta eficiencia
INSERT INTO producto (codigo_producto, nombre, cantidad_en_stock,
precio_venta)
VALUES
('PROD-P01', 'Pala de Acero Reforzada', 150, 19.95),
('PROD-M02', 'Manguera Autoenrollable 20m', 80, 24.50),
('PROD-T03', 'Tijeras de Podar Premium', 45, 12.99);
```

Sentencia INSERT ... SELECT

Permite poblar una tabla de forma masiva utilizando el conjunto de resultados devuelto por una consulta SELECT. Es un recurso de rendimiento clave para migraciones de datos o generación de tablas de agregación histórica:

```
-- Supongamos una tabla para almacenar un histórico de clientes
premium
CREATE TABLE clientes_vip (
id_vip INT UNSIGNED PRIMARY KEY,
nombre VARCHAR(50) NOT NULL,
credito DECIMAL(15, 2) NOT NULL
) ENGINE=InnoDB;
-- Copia selectiva de registros masivos
INSERT INTO clientes_vip (id_vip, nombre, credito)
SELECT codigo_cliente, nombre_cliente, limite_credito
FROM cliente
WHERE limite_credito >= 5000.00;
```

2.2 Modificación de Datos: Sentencia UPDATE

La sentencia UPDATE permite modificar los valores de una o más columnas de filas ya existentes en una tabla.

🔗 **Documentación Oficial: Más información en Documentación Oficial de MySQL sobre la Sentencia UPDATE**

Sintaxis de Modificación Selectiva

La cláusula WHERE es obligatoria para acotar el impacto del cambio. Su omisión modificará todos los registros de la tabla de forma irreversible:

```
-- Actualización selectiva de un producto específico
UPDATE producto
SET cantidad_en_stock = cantidad_en_stock - 5,
precio_venta = 22.50
WHERE codigo_producto = 'PROD-P01';
-- Actualización múltiple de estados basada en condiciones de
fecha
UPDATE pedido
SET estado = 'Retrasado'
WHERE fecha_entrega IS NULL
AND fecha_esperada < CURDATE();
```

⚠️ **Advertencia de Administrador:** En entornos de producción, es recomendable habilitar la variable de entorno $sql_safe_updates$ (SET sql_safe_updates = 1;). Esto previene la ejecución de sentencias UPDATE o DELETE que no utilicen un índice o clave primaria en la cláusula WHERE.

2.3 Eliminación de Datos: DELETE y TRUNCATE

La eliminación de registros puede realizarse de forma lógica, física selectiva o mediante el vaciado completo de la estructura física.

🔗 **Documentación Oficial: Más información en Documentación Oficial de MySQL sobre la Sentencia DELETE**

🔗 **Documentación Oficial: Más información en Documentación Oficial de MySQL sobre la Sentencia TRUNCATE TABLE**

Sentencia DELETE (Selectivo)

Borra los registros que cumplen las condiciones del filtro WHERE uno a uno, disparando disparadores (triggers) y registrando la operación en el registro binario (binary log):

```
-- Borrado selectivo de un pago específico
DELETE FROM pago
WHERE id_transaccion = 'TXN-2026-001';
```

Sentencia TRUNCATE (Borrado de Alta Eficiencia)

Es una sentencia DDL que, en lugar de eliminar fila por fila, destruye la tabla físicamente y la recrea vacía desde cero.

- Es extremadamente rápida en tablas con millones de registros.
- No admite cláusula WHERE.
- Libera de forma inmediata el espacio en disco asignado a la tabla.
- **No** activa los desencadenadores de borrado (ON DELETE TRIGGERS).
- Reinicia los contadores auto-incrementales (AUTO_INCREMENT = 1).
- No puede ejecutarse en tablas con claves externas activas referenciadas por otras tablas.

```
-- Vaciar por completo los datos históricos para preparar una nueva
carga de datos
TRUNCATE TABLE clientes_vip;
```

2.4 Operaciones de Manipulación e Integridad Referencial

Cuando modificamos o eliminamos registros que están asociados con otras tablas a través de relaciones de clave ajena (FOREIGN KEY), el SGBD actúa según la regla referencial definida:

```
-- Intento de borrado de un producto referenciado
DELETE FROM producto WHERE codigo_producto = 'PROD-M02';
```

ON DELETE RESTRICT / NO ACTION (Por defecto): El motor InnoDB de MySQL detiene la sentencia y lanza un error si el código del producto existe en alguna fila de la tabla detalle_pedido.

```
ON DELETE CASCADE: Si se borrara un registro en la
tabla pedido, InnoDB eliminará automáticamente todas las líneas
asociadas en detalle_pedido para mantener la base de datos libre de
registros huérfanos.
```

```
ON DELETE SET NULL: Si el registro principal se
elimina, los registros secundarios asociados cambian el valor de su
clave ajena a NULL (requiere que el campo acepte nulos).
```

3. TRANSACCIONES

3.1 Concepto de Transacción y Propiedades ACID

Una **transacción** es una unidad lógica de trabajo que agrupa un conjunto de sentencias SQL que deben ejecutarse como un bloque único. Para asegurar que los datos permanezcan íntegros, el motor relacional (en nuestro caso, InnoDB) debe implementar de forma estricta las propiedades **ACID**:

- **Atomicidad (Atomicity):** Garantiza el principio del "todo o nada". Si una sola de las sentencias del bloque transaccional falla, todo el bloque se descarta y los datos regresan

a su estado original.

- ```
  Consistencia (Consistency): La transacción debe
  llevar a la base de datos de un estado válido a otro, respetando
  estrictamente todas las restricciones declaradas en el esquema (claves
  primarias, de unicidad, nulos y checks).
  ```

- **Aislamiento (Isolation):** Asegura que las operaciones que ocurren dentro de una transacción en curso no sean visibles ni interfieran con otras transacciones concurrentes del sistema hasta que se consoliden.

- **Durabilidad (Durability):** Una vez que la transacción ha sido confirmada, sus modificaciones se vuelven persistentes en el almacenamiento físico, soportando fallos de alimentación del servidor, caídas del sistema operativo o pérdidas de red de los clientes.

3.2 Control del AUTOCOMMIT en MySQL

Por defecto, MySQL opera con la variable de entorno $autocommit$ activa (valor 1). Esto significa que cada comando DML individual (INSERT, UPDATE, DELETE) se trata como una transacción independiente y se escribe de forma automática y definitiva en disco.

🔗 **Documentación Oficial: Información sobre Autocommit en** [Documentación Oficial de MySQL sobre Autocommit]

Para trabajar de manera profesional con transacciones complejas, debemos configurar y monitorizar el estado de este parámetro en el servidor:

```
-- Consultar el estado del autocommit de la sesión actual
SELECT @@SESSION.autocommit;
-- Desactivar el autocommit (Recomendado para scripts interactivos de
SysAdmin)
SET autocommit = 0;
-- Volver a activar el autocommit automático
SET autocommit = 1;
```

3.3 Comandos TCL: START TRANSACTION, COMMIT y ROLLBACK
```

Los comandos del Lenguaje de Control de Transacciones (TCL) permiten delimitar el alcance del bloque lógico de trabajo.

🔗 **Documentación Oficial: Más Información en Documentación de START TRANSACTION, COMMIT y ROLLBACK**

START TRANSACTION / BEGIN

Inicia un bloque transaccional de forma explícita. Suspende temporalmente el comportamiento del autocommit si estuviera activo para la sesión de trabajo.

COMMIT

Guarda de forma permanente y definitiva todas las modificaciones realizadas durante la transacción activa en el almacenamiento físico del servidor.

ROLLBACK

Aborta la transacción activa, revierte todas las modificaciones en memoria y devuelve los datos a su estado original exacto (el estado inmediatamente anterior a la instrucción START TRANSACTION).

Ejemplo Práctico Integrado (Flujo de Pedido)

A continuación se ilustra un proceso real donde un cliente realiza un pedido de un producto. Esta operación requiere:

1. Validar que hay stock.
2. Crear el pedido.
3. Descontar el stock de la tabla producto.
4. Generar la línea de detalle.

Si falla la inserción de las líneas de detalle, se debe abortar la operación para no descontar existencias de forma errónea:

```
-- Iniciamos explícitamente el bloque transaccional
START TRANSACTION;
-- Paso 1: Insertar la cabecera del pedido para el cliente 1
INSERT INTO pedido (fecha_pedido, fecha_esperada, estado,
codigo_cliente)
```

```
VALUES (CURDATE(), DATE_ADD(CURDATE(), INTERVAL 7 DAY), 'Pendiente',
1);
-- Guardamos el ID autoincremental recién generado para el pedido
SET @nuevo_pedido = LAST_INSERT_ID();
-- Paso 2: Restar el stock en la tabla de productos (Pala de
Acero)
UPDATE producto
SET cantidad_en_stock = cantidad_en_stock - 2
WHERE codigo_producto = 'PROD-P01';
-- Paso 3: Registrar la línea detallada del pedido
INSERT INTO detalle_pedido (codigo_pedido, codigo_producto, cantidad,
precio_unidad)
VALUES (@nuevo_pedido, 'PROD-P01', 2, 19.95);
-- Confirmar cambios
COMMIT;
```

Si durante el script se detecta que no se cumple la restricción referencial o el stock es insuficiente, se puede invocar en cualquier punto antes del COMMIT:

```
-- En caso de error, abortar todas las operaciones del bloque
ROLLBACK;
```

3.4 Control Fino con Puntos de Retorno (SAVEPOINT)

Los puntos de retorno (SAVEPOINT) permiten dividir una transacción extensa en etapas secundarias. De esta forma, el administrador puede realizar retrocesos parciales de la información sin desechar todo el trabajo realizado en la transacción.

🔗 **Documentación Oficial: Más Información en Documentación Oficial de MySQL sobre SAVEPOINT**

```
START TRANSACTION;
-- Registro de la cabecera del pedido
INSERT INTO pedido (fecha_pedido, fecha_esperada, estado,
codigo_cliente)
VALUES (CURDATE(), DATE_ADD(CURDATE(), INTERVAL 15 DAY), 'Pendiente',
1);
SET @pedido_id = LAST_INSERT_ID();
-- Definimos el primer punto de control
SAVEPOINT sp_cabecera_pedido;
-- Añadir primer producto al detalle
INSERT INTO detalle_pedido (codigo_pedido, codigo_producto, cantidad,
```

```
precio_unidad)
VALUES (@pedido_id, 'PROD-P01', 1, 19.95);
-- Definimos el segundo punto de control
SAVEPOINT sp_primer_producto;
-- Añadir un segundo producto que podría fallar
INSERT INTO detalle_pedido (codigo_pedido, codigo_producto, cantidad,
precio_unidad)
VALUES (@pedido_id, 'PROD-X99', 5, 49.99);
-- Regresar al estado sp_primer_producto en caso de fallo
ROLLBACK TO SAVEPOINT sp_primer_producto;
-- Liberar punto de retorno y confirmar
RELEASE SAVEPOINT sp_cabecera_pedido;
COMMIT;
```

4. ACCESO CONCURRENTE Y NIVELES DE AISLAMIENTO EN INNODB

En servidores multiusuario modernos, cientos de conexiones de red pueden intentar interactuar de forma simultánea con las mismas tablas. Esto da lugar a una serie de fenómenos de lectura y actualización concurrente que deben gestionarse de forma estricta.

4.1 Fenómenos de Lectura Concurrente

1. Lectura Sucia (Dirty Read)

Ocurre cuando la **Transacción A** modifica una fila, y la **Transacción B** lee esa fila modificada *antes* de que la Transacción A haga un COMMIT. Si la Transacción A decide realizar un ROLLBACK, la Transacción B habrá recuperado y trabajado con un dato irreal que nunca se consolidó en el sistema.

Transacción A Transacción B
| |
|-- UPDATE producto |
| SET precio_venta = 50.00 |
| WHERE id="a29" |
| |-- SELECT precio_venta (Lee 50.00)
| | *(Lectura sucia)*
|-- ROLLBACK (Fallo en el sistema) |

2. Lectura No Repetible (Non-Repeatable Read)

Ocurre cuando una transacción lee los datos de una fila y, en el transcurso de la misma transacción, vuelve a leer el mismo registro pero encuentra que los datos han sido modificados o eliminados por otra transacción externa que ha hecho un COMMIT en ese intervalo de tiempo.

Transacción A Transacción B
START TRANSACTION;
SELECT precio FROM producto

WHERE id=1; -- Devuelve 10.00
START TRANSACTION;
UPDATE producto SET precio = 15.00 WHERE id = 1;
COMMIT;
SELECT precio FROM producto

WHERE id=1; -- ¡Devuelve 15.00!

El mismo SELECT da resultados distintos en la misma transacción.

3. Lectura Fantasma (Phantom Read)

Ocurre cuando una transacción ejecuta una consulta de rango de filas (ej: un SELECT condicionado), y una transacción concurrente inserta o elimina registros en ese rango y realiza un COMMIT. Al repetir la consulta inicial dentro de la primera transacción, se observa un conjunto de filas diferente (los registros "fantasma").

Transacción A Transacción B
START TRANSACTION;
SELECT * FROM producto
WHERE precio_venta

BETWEEN 10 AND 50;
-- Devuelve 2 filas (A y B)
START TRANSACTION;
INSERT INTO producto VALUES (3, 'Prod C', 25.00);
COMMIT;
SELECT * FROM producto
WHERE precio_base

```
BETWEEN 10 AND 50;
-- ¡Devuelve 3 filas! (A, B y el fantasma C)
```

4.2 Niveles de Aislamiento

El estándar SQL define cuatro **niveles de aislamiento** para controlar de manera fina el equilibrio entre consistencia y rendimiento del sistema. A mayor aislamiento, menor probabilidad de fallos de concurrencia, pero menor rendimiento debido al bloqueo físico de recursos.

🔗 **Documentación Oficial: Más Información en [Documentación de Niveles de Aislamiento de Transacciones en InnoDB]**

Nivel de Aislamiento	Lectura Sucia	Lectura No Repetible	Lectura Fantasma
READ UNCOMMITTED	Permitido	Permitido	Permitido
READ COMMITTED	Evitado	Permitido	Permitido
REPEATABLE READ (Por defecto InnoDB)	Evitado	Evitado	Evitado en MySQL*
SERIALIZABLE	Evitado	Evitado	Evitado

Nota técnica de administración: En la mayoría de sistemas SQL estándar, el nivel REPEATABLE READ permite lecturas fantasma. Sin embargo, el motor InnoDB de MySQL previene de forma nativa la inserción de registros fantasma utilizando la técnica de bloqueo **Next-Key Locking.*

Gestión de Variables de Aislamiento en MySQL

Como administradores, podemos consultar y parametrizar los niveles de aislamiento a nivel de servidor (GLOBAL) o a nivel de conexión activa (SESSION):

```
-- Consultar el aislamiento a nivel de servidor global
SELECT @@GLOBAL.transaction_isolation;
-- Consultar el aislamiento de la sesión activa
SELECT @@SESSION.transaction_isolation;
-- Consultar de forma simplificada
SELECT @@transaction_isolation;
-- Modificar el nivel de aislamiento de la sesión
```

```
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

5. TRANSACCIONES EN PROCEDIMIENTOS ALMACENADOS

En la arquitectura de sistemas real, no es aconsejable delegar el control de transacciones de negocio crítico al cliente externo (PHP, Python, etc.). Es una práctica recomendada centralizar las transacciones complejas dentro del propio gestor mediante **Procedimientos Almacenados (Stored Procedures)**.

🔗 **Documentación Oficial: Más Información en Documentación de MySQL sobre el control de flujo y excepciones en Stored Procedures**

5.1 Gestión de Excepciones y Reversión Automática

El motor de MySQL permite declarar manejadores de errores (DECLARE HANDLER). En caso de que ocurra una violación de restricción física (error de clave externa, de tipo o nulo), el manejador ejecutará una sección de código encargada de hacer el rollback automático de forma controlada y segura para la sesión:

```
DELIMITER $$
CREATE PROCEDURE procesar_pago_pedido (
IN p_cliente_id INT UNSIGNED,
IN p_transaccion VARCHAR(26),
IN p_monto DECIMAL(15, 2)
)
BEGIN
DECLARE t_error_detectado INT DEFAULT 0;
DECLARE CONTINUE HANDLER FOR SQLEXCEPTION
BEGIN
SET t_error_detectado = 1;
ROLLBACK;
END;
START TRANSACTION;
INSERT INTO pago (codigo_cliente, id_transaccion, fecha_pago,
total)
VALUES (p_cliente_id, p_transaccion, CURDATE(), p_monto);
UPDATE cliente
SET limite_credito = limite_credito + (p_monto * 0.05)
```

```
WHERE codigo_cliente = p_cliente_id;
IF t_error_detectado = 0 THEN
COMMIT;
SELECT 'Transacción procesada correctamente.' AS Estado;
ELSE
SELECT 'Error crítico. Transacción abortada.' AS Estado;
END IF;
END$$
DELIMITER ;
```